

Multi-Asset Stochastic Pricing & XVA Risk Engine

Documentation: Technical Architecture, Mathematical Foundations, and Implementation

Developed by Udayveer Singh Andotra

September 2026

Contents

1	Introduction, Motivation, and Real-World Scenario	2
1.1	The Regulatory and Market Context	2
1.2	Project Motivation and Scenario Description	2
2	System Architecture & Codebase Structure	3
3	Phase 1: High-Performance Multi-Asset Simulation Engine	4
3.1	Geometric Brownian Motion & The Euler-Maruyama Discretization	4
3.1.1	Itô's Lemma and Log-Normal Path Updating	4
3.2	The Heston Model (Stochastic Volatility)	4
3.2.1	Full Truncation Numerical Scheme	4
3.3	Merton Jump-Diffusion & Market Crashes	5
3.4	Cholesky Decomposition & Spectral Regularization	5
3.5	Vasicek Interest Rate Model	5
4	Phase 2: Macroeconomic Yield Curve Bootstrapper	6
4.1	Bootstrapping Mechanics	6
4.2	Cubic Splines vs. Nelson-Siegel-Svensson (NSS)	6
5	Phase 3: Counterparty Credit Risk, Exposure Profiling, and XVA Analytics	7
5.1	Mathematical Formulation of Exposure Metrics	7
5.2	Credit Value Adjustment (CVA) & Funding Value Adjustment (FVA)	7
5.3	Wrong-Way Risk (WWR) & Numerical Overflow Mitigation	7
6	Phase 4: Hybrid Portfolio Optimization and Production Deployment	8
6.1	The Multi-Objective Risk Score Equation and Mathematical Rationale for Weights (15 and 8)	8
6.1.1	Derivation and Quantitative Rationale for Factors 15.0 and 8.0	8
6.2	The Zero-Exposure Trap and Growth Penalty	8
6.3	Hybrid Optimization Solver Strategy	9
7	Interactive Dashboard Architecture: The Streamlit API	10
7.1	Core Dashboard Components	10
8	Future Prospects and Practical Applications	10
8.1	Industry and Practical Applications	10
8.2	Future Roadmap and Architectural Extensions	10

1 Introduction, Motivation, and Real-World Scenario

1.1 The Regulatory and Market Context

Following the 2008 global financial crisis, the structural architecture of over-the-counter (OTC) derivatives markets underwent a permanent paradigm shift [4]. Historically, trading desks priced derivative contracts under a risk-free assumption, treating counterparty default risk as a negligible externality. The realization that major financial institutions could default exposed systemic vulnerabilities, leading international regulators (Basel III/IV) to mandate comprehensive framework measurements for **Counterparty Credit Risk (CCR)** and **Valuation Adjustments (XVA)** [1].

In modern quantitative finance, quoting a derivative price without accounting for Credit Value Adjustment (CVA) or Funding Value Adjustment (FVA) is commercially unviable. Banks must dynamically quantify the expected future cost of counterparty default and collateral funding across multi-year horizons [4].

1.2 Project Motivation and Scenario Description

The motivation for this project is to construct a production-grade, multi-asset quantitative pricing and risk engine from the ground up [1]. Standard academic models often rely on simplistic assumptions—such as constant volatility, linear interest rates, and risk-free counterparties—which fail during systemic market stress.

The Operational Scenario: Consider a tier-1 bank trading a complex multi-asset basket option or uncollateralized derivative portfolio mapped across 20 distinct liquid equities and indices (e.g., major NSE equities and ETFs). The risk management division requires an automated system capable of:

1. Simulating thousands of correlated, fat-tailed stochastic market paths under varying macroeconomic stress scenarios (Baseline, Bullish, and Market Crash) [1].
2. Bootstrapping and modeling continuous macroeconomic discount structures to present-value future cash flows [4].
3. Computing dynamic counterparty risk exposures, including Expected Exposure (EE), 95% Potential Future Exposure (PFE), CVA, and FVA [4].
4. Executing hybrid global-local portfolio optimizations to reallocate capital, minimizing regulatory capital charges while enforcing strict structural portfolio growth [1].

2 System Architecture & Codebase Structure

To maintain clean separation of concerns and ensure high-performance execution, the engine is organized into a modular, production-ready directory structure:

Module Path	Architectural Responsibility
<code>core/monte_carlo.py</code>	High-performance simulation engine. Handles GPU/MPS-accelerated tensor operations, Cholesky decomposition with spectral clipping, Heston stochastic volatility, and Merton jump-diffusion [1, 2].
<code>core/yield_curve.py</code>	Macroeconomic discounting module. Implements zero-rate yield curve bootstrapping and Nelson-Siegel-Svensson (NSS) continuous curve fitting [4].
<code>core/xva_calculator.py</code>	Counterparty credit risk analytics module. Evaluates derivative payoffs (Calls/Puts), computes EE, 95% PFE, CVA, FVA, and handles Wrong-Way Risk (WWR) scaling [4].
<code>engine_backend.py</code>	Orchestration pipeline. Fetches historical market data via <code>yfinance</code> , computes empirical drifts/covariances, executes scenario modifications, and runs the <code>differential_evolution</code> and <code>SLSQP</code> optimizers [6].
<code>app.py</code>	Interactive Streamlit frontend dashboard. Manages session state, user input parameters, real-time logging status containers, and persistent execution rendering [7].

Table 1: System Directory Structure and Module Mapping

3 Phase 1: High-Performance Multi-Asset Simulation Engine

The foundation of any quantitative risk system is its stochastic simulation core. Before pricing derivatives or measuring credit risk, the engine must generate realistic future market states [1].

3.1 Geometric Brownian Motion & The Euler-Maruyama Discretization

The continuous-time stochastic differential equation (SDE) governing standard Geometric Brownian Motion (GBM) is defined as [1]:

$$dS_t = \mu S_t dt + \sigma S_t dW_t \quad (1)$$

where μ represents the deterministic drift, σ is the volatility, and dW_t is the standard Wiener process increment. Because computers operate in discrete time, continuous SDEs must be approximated using numerical discretization schemes [1]. Applying the Euler-Maruyama method yields:

$$S_{t+\Delta t} = S_t + \mu S_t \Delta t + \sigma S_t \sqrt{\Delta t} Z \quad (2)$$

where Δt is the discrete time step size (e.g., $\frac{1}{252}$ for daily trading steps) and $Z \sim \mathcal{N}(0, 1)$ is a standard normal random shock [1].

3.1.1 Itô's Lemma and Log-Normal Path Updating

While Equation 2 provides an arithmetic approximation, simulating exponential asset paths additively over multi-year horizons introduces a severe downward discretization bias. To eliminate this, the engine applies Itô's Lemma to the log-price transformation $X_t = \ln(S_t)$:

$$dX_t = \left(\mu - \frac{1}{2} \sigma^2 \right) dt + \sigma dW_t \quad (3)$$

Discretizing Equation 3 exactly via Euler-Maruyama produces the log-normal path update implemented in the core engine:

$$\ln S_{t+\Delta t} = \ln S_t + \left(\mu - \frac{1}{2} \sigma^2 \right) \Delta t + \sigma \sqrt{\Delta t} Z \quad (4)$$

Exponentiating this result ($S_{t+\Delta t} = \exp(\ln S_{t+\Delta t})$) guarantees absolute price positivity and eliminates arithmetic drift bias [1].

3.2 The Heston Model (Stochastic Volatility)

Constant volatility models fail to capture volatility clustering and leverage effects observed during market crises. The Heston model introduces a secondary stochastic process for variance (v_t) governed by the Cox-Ingersoll-Ross (CIR) framework [2]:

$$dS_t = \mu S_t dt + \sqrt{v_t} S_t dW_{1,t} \quad (5)$$

$$dv_t = \kappa(\theta - v_t)dt + \xi \sqrt{v_t} dW_{2,t} \quad (6)$$

where κ is the mean-reversion speed, θ is the long-term variance, ξ is the volatility of volatility ("vol-of-vol"), and $dW_{1,t}$ and $dW_{2,t}$ are correlated Wiener processes such that $\mathbb{E}[dW_{1,t} dW_{2,t}] = \rho dt$ [2].

3.2.1 Full Truncation Numerical Scheme

Discrete Euler steps can push variance below zero, causing fatal NaN errors when evaluating $\sqrt{v_t}$ [1]. To resolve this, the engine implements the **Full Truncation** scheme [1]:

$$v_{t+\Delta t} = \max \left(v_t + \kappa(\theta - v_t) \Delta t + \xi \sqrt{\max(v_t, 0)} \sqrt{\Delta t} Z_2, 10^{-6} \right) \quad (7)$$

3.3 Merton Jump-Diffusion & Market Crashes

To simulate sudden market gaps (e.g., earnings shocks or systemic crashes), a compound Poisson jump process is superimposed onto the continuous diffusion [1]. Jumps arrive according to a Poisson process with intensity λ , where jump sizes are normally distributed $J \sim \mathcal{N}(\mu_j, \sigma_j^2)$. To preserve the objective drift μ , the engine incorporates a martingale jump compensator λk where $k = \mathbb{E}[e^J - 1] = \exp\left(\mu_j + \frac{1}{2}\sigma_j^2\right) - 1$ [1].

3.4 Cholesky Decomposition & Spectral Regularization

To simulate a basket of 20 correlated assets, random shocks are correlated using a historical covariance matrix Σ via Cholesky decomposition ($LL^T = \Sigma$) [1]. Because empirical high-dimensional covariance matrices often suffer from ill-conditioning and negative eigenvalues, the engine applies **Spectral Eigenvalue Clipping**: computing eigenvalues and shifting the diagonal by ϵI if positive-definiteness is threatened [1].

3.5 Vasicek Interest Rate Model

Interest rates operate in banded environments governed by central bank policy. The Vasicek model captures this via an Ornstein-Uhlenbeck mean-reverting process [3]:

$$dr_t = a(b - r_t)dt + \sigma_r dW_t \quad (8)$$

where a is the reversion speed, b is the long-term mean level, and σ_r is the short-rate volatility [3].

4 Phase 2: Macroeconomic Yield Curve Bootstrapper

Derivatives pricing requires discounting future cash flows to present value using a continuous Zero-Coupon Yield Curve [4].

4.1 Bootstrapping Mechanics

Bootstrapping isolates pure zero-coupon rates by algebraically stripping intermediate coupon payments from liquid market instruments (e.g., deposits, swaps) [4]. Given a set of market instruments with cash flows C_{ij} occurring at times t_j and market prices P_i , the discount factor $Z(t_j)$ satisfies:

$$P_i = \sum_{j=1}^m C_{ij} Z(t_j) \quad (9)$$

Solving iteratively extracts the zero rates $r(t_j) = -\frac{\ln Z(t_j)}{t_j}$ [4].

4.2 Cubic Splines vs. Nelson-Siegel-Svensson (NSS)

Basic linear interpolation creates jagged angles, resulting in unstable risk sensitivities ("Greeks") [4]. While Cubic Splines smooth this out using piecewise third-degree polynomials with natural boundary conditions (*bc_type = 'natural'*), they are prone to the Runge phenomenon (oscillations at curve endpoints) [4]. For macro-stable curves, the engine also supports the parametric **Nelson-Siegel-Svensson (NSS)** model [4]:

$$y(t) = \beta_0 + \beta_1 \left(\frac{1 - e^{-t/\tau_1}}{t/\tau_1} \right) + \beta_2 \left(\frac{1 - e^{-t/\tau_1}}{t/\tau_1} - e^{-t/\tau_1} \right) + \beta_3 \left(\frac{1 - e^{-t/\tau_2}}{t/\tau_2} - e^{-t/\tau_2} \right) \quad (10)$$

5 Phase 3: Counterparty Credit Risk, Exposure Profiling, and XVA Analytics

Phase 3 bridges risk-free derivative pricing with real-world counterparty default realities [4].

5.1 Mathematical Formulation of Exposure Metrics

Credit risk materializes exclusively when a derivative’s Mark-to-Market (MtM) value V_t is positive to the lending institution [4].

- **Positive Exposure E_t :** Defined as $E_t = \max(V_t, 0)$ [4]. For European Call options, $V_t = \max(S_t - K, 0)$; for Put options, $V_t = \max(K - S_t, 0)$ [4].
- **Expected Exposure (EE):** The cross-sectional statistical mean of positive exposures across all Monte Carlo paths at time step t_i [4]:

$$EE(t_i) = \frac{1}{M} \sum_{m=1}^M E_m(t_i) \quad (11)$$

- **Potential Future Exposure (PFE):** The 95th percentile of positive exposures at each time step, representing worst-case credit risk thresholds used for credit limit allocation [4]:

$$PFE_{95}(t_i) = \text{Percentile}(\{E_m(t_i)\}_{m=1}^M, 95\%) \quad (12)$$

5.2 Credit Value Adjustment (CVA) & Funding Value Adjustment (FVA)

CVA represents the expected loss due to counterparty default over the life of the trade [4]:

$$CVA = (1 - R) \sum_{i=1}^N EE(t_i) \cdot DF(t_i) \cdot PD(t_i) \quad (13)$$

where R is the recovery rate (e.g., 40

$$PD(t_i) = e^{-\lambda t_{i-1}} - e^{-\lambda t_i} \quad (14)$$

Funding Value Adjustment (FVA) captures the cost of funding uncollateralized derivative positions through the corporate treasury [4]:

$$FVA = \sum_{i=1}^N s_{fund} \cdot EE(t_i) \cdot DF(t_i) \cdot \Delta t_i \quad (15)$$

where s_{fund} is the funding spread [4].

5.3 Wrong-Way Risk (WWR) & Numerical Overflow Mitigation

Wrong-Way Risk occurs when exposure correlates adversely with the counterparty’s credit quality [4]. WWR is modeled by making the hazard rate stochastic and dependent on portfolio exposure [4]:

$$\lambda_t = \lambda_{base} \exp(\alpha \cdot E_t) \quad (16)$$

Numerical Safeguard: In currency scales (e.g., INR 10,000,000+), multiplying absolute exposure by α produces massive exponents, causing `torch.exp()` to overflow to infinity (inf). The engine resolves this by normalizing exposure relative to initial capital (E_t/V_0) and clamping the exponent to a maximum of 20.0, guaranteeing numerical stability.

6 Phase 4: Hybrid Portfolio Optimization and Production Deployment

Phase 4 implements a quantitative optimization layer to reallocate portfolio weights, minimizing regulatory XVA charges while preserving economic viability [1].

6.1 The Multi-Objective Risk Score Equation and Mathematical Rationale for Weights (15 and 8)

The optimization routine searches for an asset weight vector w that minimizes aggregate capital risk charges according to the objective risk score function [6]:

$$\text{Risk Score}(w) = \max_t PFE_{95}(t) + 15.0 \cdot CVA + 8.0 \cdot FVA + \text{Growth Penalty} \quad (17)$$

6.1.1 Derivation and Quantitative Rationale for Factors 15.0 and 8.0

Because the weights 15.0 and 8.0 act as calibration and scaling multipliers rather than closed-form analytical constants, their mathematical derivation stems from **unit harmonization**, **order-of-magnitude scaling**, and **regulatory capital weighting**:

- **Unit Harmonization and Magnitude Scaling:** Peak PFE ($\max_t PFE_{95}(t)$) is evaluated as a large, absolute currency threshold (e.g., millions representing worst-case portfolio exposure). Conversely, CVA and FVA are calculated as present-value integrals over time involving fractional default probabilities (PD) and funding spreads ($\sim 1\%$), making raw CVA and FVA values one to two orders of magnitude smaller than peak PFE. Without scaling multipliers, the optimizer would treat CVA/FVA as negligible noise and optimize solely for PFE.
- **Regulatory Capital Mapping (15.0 for CVA):** Under regulatory capital frameworks (Basel III), CVA volatility and unexpected default risk carry heavy capital charge multipliers [4]. The factor of 15.0 scales the present-value CVA expected loss into an equivalent capital-at-risk metric, aligning its gradient weight with regulatory capital penalties.
- **Treasury Funding Weighting (8.0 for FVA):** FVA represents ongoing treasury funding costs rather than sudden default insolvency tail-losses [4]. A weight of 8.0 appropriately scales its priority relative to peak credit exposure (PFE) and credit default risk (CVA) while ensuring active management of funding liquidity drag.

6.2 The Zero-Exposure Trap and Growth Penalty

A critical quantitative trap encountered during development was the **Zero-Exposure Trap**: when optimizing call option portfolios ($\max(S_t - K, 0)$), an optimizer tasked solely with minimizing credit risk quickly discovered that if portfolio weights drove asset values downward so that they never breached strike K , exposure remained exactly zero, eliminating CVA and PFE.

To prevent this yield destruction, the engine embeds a dynamic **Growth Penalty**:

$$\text{Growth Penalty} = \max(0, V_0 - \mathbb{E}[V_T]) \cdot 5.0 \quad (18)$$

If the expected terminal portfolio value falls below initial capital V_0 , the optimizer incurs a heavy penalty, forcing it to achieve low risk through intelligent diversification rather than asset annihilation.

6.3 Hybrid Optimization Solver Strategy

The optimization pipeline employs a two-stage hybrid solver via SciPy [6]: 1. **Global Optimization** (`differential_evolution`): Explores the non-convex parameter space across bounds $w_i \in [0, 0.25]$ to locate the global basin of attraction, avoiding local minima. 2. **Local Refinement** (`SLSQP`): Takes the global solution as a starting guess and applies Sequential Least Squares Programming under equality constraints ($\sum w_i = 1.0$) to achieve precise convergence.

7 Interactive Dashboard Architecture: The Streamlit API

To provide a production-grade interface for risk managers and quantitative researchers, the engine is paired with an interactive web frontend built using the **Streamlit API** (`app.py`) [7].

7.1 Core Dashboard Components

- **Session State Persistence (`st.session_state`):** Manages multi-page application states, scenario toggling (Baseline, Bullish, Market Crash), and user-defined initial capital parameters across asynchronous runs.
- **Real-Time Status Logging Containers:** Utilizes dynamic status blocks (`st.status`) to stream progress updates across asynchronous execution phases—from `yfinance` historical data ingestion to GPU simulation, NSS yield curve fitting, and global-local optimization [6, 9].
- **Automated Matplotlib Figure Integration:** Renders multi-panel risk profile slides directly into the web UI via `st.pyplot()`, displaying Phase 1 asset paths, Phase 3 exposure profiles (EE and PFE curves), and Phase 2 zero-coupon yield curves side-by-side [8].

8 Future Prospects and Practical Applications

8.1 Industry and Practical Applications

The multi-asset stochastic pricing and XVA engine transitions theoretical quantitative concepts into production tools used across modern financial institutions [1, 4]:

- **XVA Trading Desks:** Quantitative analysts and front-office traders use real-time exposure simulations to price CVA, FVA, and related adjustments into complex OTC derivative structures before client execution [4].
- **Risk Methodology & Credit Risk Management:** Chief Risk Officers (CROs) utilize PFE profile boundaries and Wrong-Way Risk analytics to dynamically establish counterparty credit limits, capital reserves, and collateral thresholds under Basel III/IV mandates [1].
- **Regulatory Capital Optimization:** Portfolio managers apply optimization layers to discover asset allocations that minimize uncollateralized credit risk charges without sacrificing baseline asset returns [1].

8.2 Future Roadmap and Architectural Extensions

To scale the engine from a portfolio-grade prototype to a tier-1 enterprise system, several advanced extensions are planned [1]:

- **Dual-Curve OIS Discounting Framework:** Upgrading the yield curve module to decouple cash flow projection curves (e.g., SOFR/LIBOR) from collateralized discounting curves (OIS) to reflect post-2008 institutional funding mechanics [4].
- **Capital Value Adjustment (KVA):** Incorporating KVA modeling to compute the lifetime cost of holding regulatory capital against uncollateralized derivative portfolios [4].

References

- [1] Glasserman, P. (2003). *Monte Carlo Methods in Financial Engineering*. Springer-Verlag.
- [2] Heston, S. L. (1993). A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options. *Review of Financial Studies*, 6(2), 327–343.
- [3] Vasicek, O. (1977). An Equilibrium Characterization of the Term Structure. *Journal of Financial Economics*, 5(2), 177–188.
- [4] Brigo, D., & Mercurio, F. (2006). *Interest Rate Models—Theory and Practice: With Smile, Inflation and Credit*. Springer Science & Business Media.
- [5] PyTorch Documentation. Tensor Operations and GPU Acceleration. <https://pytorch.org/>
- [6] SciPy Developers. Interpolation and Optimization Modules (`scipy.interpolate`, `scipy.optimize`). <https://scipy.org/>
- [7] Streamlit Inc. Streamlit API for Interactive Data Applications. <https://streamlit.io/>
- [8] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
- [9] Ran Aroussi. Yahoo! Finance market data downloader (`yfinance`). <https://github.com/ranaroussi/yfinance>